

Universidad Nacional Autónoma de México

Facultad de Ingeniería, Ingeniería en Computación.



Arquitectura Cliente-Servidor.

Examen Extraordinario. SISTEMA MANEJADOR DE BASES DE DATOS.

PROFESOR: MTRO. CARLOS ALBERTO ROMÁN ZAMITIZ

Semestre: 2027-1

Fecha de entrega: 1/Noviembre/2026

Alumno: Arreola Santander Ricardo

ÍNDICE

INTRODUCCIÓN 3
- Arquitectura cliente-servidor 3
- Sockets TCP/IP 3
- Sistema manejador de bases de datos 4
- Lenguaje SQL 5

OBJETIVOS 6

DESARROLLO 7
- Estructura del proyecto 7
- Constantes compartidas 7
- Funciones de apoyo 8
- Servidor 9
- Cliente 13
- Manejo de la tabla 16
- Intérprete de comandos SQL 21
- Makefile 30

RESULTADOS 31

CONCLUSIONES 37

BIBLIOGRAFÍA Y FUENTES 38

INTRODUCCIÓN

Un **sistema manejador de bases de datos** (DBMS por sus siglas en inglés) es un programa que se encarga de guardar, consultar y modificar información de forma organizada. El usuario no trabaja directamente con los archivos en disco, sino que le envía instrucciones al manejador y éste se encarga de leer o escribir los datos y de regresar el resultado.

En este proyecto se construye un manejador sencillo en el que la base de datos tiene una sola tabla, guardada en un archivo de texto, y en el que el manejador se divide en dos programas que corren en computadoras distintas: un servidor, que tiene el archivo y ejecuta las instrucciones, y un cliente, que se conecta al servidor a través de la red para enviarle las instrucciones y mostrar las respuestas.

Arquitectura cliente-servidor

La **arquitectura cliente-servidor** es una forma de organizar una aplicación en dos partes que se comunican por medio de una red:

1. **Servidor:** Es el programa que ofrece un servicio. Permanece en ejecución esperando a que lleguen peticiones, las atiende y regresa una respuesta. En este proyecto el servidor es quien tiene el archivo de la tabla y quien ejecuta los comandos.
2. **Cliente:** Es el programa que utiliza el servicio. Se conecta al servidor, le envía peticiones y muestra al usuario las respuestas que recibe. En este proyecto el cliente es la terminal en la que el usuario escribe los comandos SQL.

Con esta organización varios clientes pueden usar el mismo servicio al mismo tiempo desde diferentes máquinas, y los datos se mantienen en un solo lugar. Para que el servidor pueda atender a varios clientes a la vez, en este proyecto se crea un proceso hijo por cada cliente que se conecta, de modo que cada conversación se lleva por separado.

Sockets TCP/IP

Un **socket** es el mecanismo que ofrece el sistema operativo para que dos programas se comuniquen a través de la red. Para el programa, un socket se maneja como un descriptor de archivo: se le escriben bytes con `send` y se le leen bytes con `recv`.

Los sockets de este proyecto son de la familia **AF_INET**, es decir, utilizan direcciones IP versión 4, y son de tipo **SOCK_STREAM**, es decir, utilizan el protocolo **TCP**. TCP es un protocolo orientado a conexión: antes de intercambiar datos, cliente y servidor establecen una conexión, y durante ella TCP garantiza que los bytes lleguen completos y en orden. A cambio, TCP no conserva los límites de cada mensaje; lo que se envía con varias llamadas

a `send` puede llegar en una sola llamada a `recv` o repartido en varias. Por eso la aplicación necesita definir cómo se separa un mensaje del siguiente.

Las funciones que se utilizan en cada lado son las siguientes:

Servidor	Cliente
`socket` crea el socket	`socket` crea el socket
`bind` lo asocia a una dirección y un puerto	`gethostbyname` obtiene la dirección del servidor
`listen` lo pone a esperar conexiones	`connect` se conecta al servidor
`accept` acepta a un cliente y regresa un nuevo socket	`send` y `recv` intercambian datos
`send` y `recv` intercambian datos	`close` cierra la conexión
`close` cierra la conexión	

Un detalle importante es el orden de los bytes. Los números de puerto se guardan en la estructura de dirección en **orden de red**, que es el byte más significativo primero. La función `htons` convierte un número del orden de la máquina al orden de red, y `ntohs` hace la conversión inversa.

Sistema manejador de bases de datos

Un manejador de bases de datos organiza la información en **tablas**. Una tabla está formada por **columnas**, que son los campos que tiene cada dato, y por **registros**, que son los renglones con los valores. En este proyecto la tabla es **Alumno** y tiene las columnas Id, Nombre, Apellido, Semestre y Carrera.

Para simplificar, la tabla se representa con un archivo de texto llamado `alumno.txt`. La primera línea del archivo es la cabecera con los nombres de las columnas y cada línea siguiente es un registro. Los campos se separan con una coma:

```
Id,Nombre,Apellido,Semestre,Carrera
1,Juan,Perez,5,Computacion
2,Anita,Lopez,8,Electronica
```

La columna **Id** es la **llave primaria**: identifica a cada registro de forma única, no se repite y no puede modificarse. Además es **autoincremental**, lo que quiere decir que el manejador la asigna de forma automática al insertar un registro, tomando el Id más grande que existe y sumándole uno.

Lenguaje SQL

SQL es el lenguaje con el que se le dan instrucciones a un manejador de bases de datos. El proyecto implementa los cuatro comandos básicos para consultar y modificar una tabla, más un comando propio para cerrar la conexión:

Comando	Ejemplo	Qué hace
SELECT	`SELECT * FROM Alumno`	Regresa todos los registros de la tabla
INSERT	`INSERT INTO Alumno VALUES (Jose,Sanchez,9,Telecom)`	Agrega un registro nuevo
UPDATE	`UPDATE Alumno SET Apellido=Gomez WHERE Id=2`	Modifica los registros que cumplen la condición
DELETE	`DELETE FROM Alumno WHERE Id=2`	Elimina los registros que cumplen la condición
SALIR	`SALIR`	Cierra la conexión con el servidor

Como lo permite el enunciado, el punto y coma al final del comando es opcional y los valores de texto no necesitan ir entre comillas.

OBJETIVOS

El alumno construirá un sistema manejador de bases de datos con arquitectura cliente-servidor, programado en lenguaje C sobre Linux, en el que el cliente se conecte de forma remota al servidor mediante sockets TCP/IP, le envíe comandos SQL y el servidor los ejecute sobre un archivo que simula una tabla.

Objetivos particulares:

- Programar un servidor que acepte conexiones remotas y atienda a varios clientes al mismo tiempo.
- Programar un cliente que se conecte al servidor desde otro host, envíe comandos y muestre las respuestas.
- Implementar los comandos SELECT, INSERT, UPDATE y DELETE sobre la tabla Alumno, respetando que la columna Id es la llave primaria y es autoincremental.
- Implementar el comando SALIR para que el cliente se desconecte.
- Probar el sistema con el cliente y el servidor en máquinas distintas.

DESARROLLO

Estructura del proyecto

El proyecto se organizó en varios archivos para separar la parte de red de la parte que maneja los datos. Cada archivo tiene una responsabilidad:

Archivo	Contenido
`src/protocolo.h`	Constantes que comparten cliente y servidor
`src/util.h`, **`src/util.c`**	Funciones de apoyo para cadenas
`src/servidor.c`	Servidor TCP, un proceso hijo por cliente
`src/cliente.c`	Cliente TCP interactivo
`src/tabla.h`, **`src/tabla.c`**	Lectura y escritura de la tabla en el archivo
`src/sql.h`, **`src/sql.c`**	Intérprete de los comandos SQL
`datos/alumno.txt`	La tabla Alumno
`Makefile`	Compilación de los dos programas

La comunicación entre cliente y servidor sigue dos reglas sencillas. El cliente envía un comando por línea, terminado con un salto de línea. El servidor responde con texto de cualquier longitud y marca el final de la respuesta con el carácter 0x04. De esta forma, aunque TCP entregue los datos en pedazos, cada lado sabe dónde termina lo que está leyendo.

El desarrollo se llevó en un repositorio de git con una rama por cada parte: servidor, cliente, SELECT y los comandos INSERT, UPDATE y DELETE. Cada rama se integró a la rama principal cuando quedó terminada.

Constantes compartidas

El archivo **`protocolo.h`** define los valores que ambos programas necesitan conocer: el puerto por omisión, el tamaño máximo de un comando y de una respuesta, el carácter que marca el fin de una respuesta y la palabra con la que se cierra la sesión.

```
1  /* Constantes que comparten el cliente y el servidor */
2  #ifndef PROTOCOLO_H
3  #define PROTOCOLO_H
4
5  #define PUERTO_DEFAULT   4898      /* puerto si no se indica otro */
6  #define TAM_PETICION     1024      /* tamanio maximo de un comando */
7  #define TAM_RESPUESTA    65536     /* tamanio maximo de una respuesta */
8  #define FIN_RESPUESTA    '\x04'    /* caracter que marca el fin de una respuesta */
9  #define CMD_SALIR        "SALIR"   /* comando para cerrar la conexion */
```

```
#define QLEN            5         /* conexiones en espera */

#endif
```

Funciones de apoyo

En **`util.c`** hay dos funciones que se usan en varias partes. **recortar_cadena** elimina los espacios y saltos de línea que hay al inicio y al final de una cadena; se usa para limpiar lo que escribe el usuario y cada campo que se lee del archivo. **quitar_comillas** elimina las comillas que encierran un valor, para que el usuario pueda escribir `Gomez` o `'Gomez'` indistintamente.

```
/* Funciones de apoyo para cadenas */
#ifndef UTIL_H
#define UTIL_H

/* Quita espacios y saltos de linea al inicio y al final de la cadena */
void recortar_cadena(char *s);

/* Quita las comillas que encierran a la cadena */
void quitar_comillas(char *s);

#endif
```

```
/* Funciones de apoyo para cadenas */
#include <ctype.h>
#include <string.h>
#include "util.h"

/* Avanza sobre los espacios del inicio, borra los del final y mueve el texto al principio
*/
void recortar_cadena(char *s)
{
    char *inicio = s;
    size_t n;

    while (*inicio != '\0' && isspace((unsigned char) *inicio))
        inicio++;

    n = strlen(inicio);
    while (n > 0 && isspace((unsigned char) inicio[n - 1]))
        inicio[--n] = '\0';

    memmove(s, inicio, n + 1);
}

/* Si el primer y el ultimo caracter son comillas, se eliminan */
void quitar_comillas(char *s)
{
    size_t n = strlen(s);

    if (n >= 2 && (s[0] == '\'' || s[0] == '"') && s[n - 1] == s[0]) {
        memmove(s, s + 1, n - 2);
        s[n - 2] = '\0';
```

```
    }
}
```

Servidor

El servidor se encuentra en **`servidor.c`**. Recibe como argumento el puerto en el que va a escuchar; si no se indica, usa el 4898. Al inicio se incluyen las cabeceras de sockets y los archivos propios del proyecto.

```
/*
 * Servidor del manejador de bases de datos.
 * Espera conexiones por TCP, recibe comandos SQL de cada cliente,
 * los ejecuta sobre el archivo de la tabla y regresa la respuesta.
 * Cada cliente se atiende en un proceso hijo.
 *
 * Uso: ./bin/servidor [puerto]
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <signal.h>
#include <time.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#include "protocolo.h"
#include "sql.h"
#include "util.h"
```

Las dos primeras funciones se encargan de enviar datos. **enviar_todo** llama a `send` en un ciclo hasta que se hayan enviado todos los bytes, porque una sola llamada puede enviar menos de los que se le piden. **enviar_respuesta** envía el texto de la respuesta y después el carácter de fin, que es la señal para que el cliente deje de leer.

```
/* Envia el buffer completo repitiendo send hasta terminar */
static int enviar_todo(int fd, const char *datos, size_t cantidad)
{
    size_t enviados = 0;
    ssize_t r;

    while (enviados < cantidad) {
        r = send(fd, datos + enviados, cantidad - enviados, 0);
        if (r <= 0)
            return -1;
        enviados += (size_t) r;
    }
    return 0;
}

/* Envia el texto y despues el caracter que marca el fin de la respuesta */
```

```
41 static int enviar_respuesta(int fd, const char *texto)
42 {
43     char fin = FIN_RESPUESTA;
44 
45     if (enviar_todo(fd, texto, strlen(texto)) != 0)
46         return -1;
47     return enviar_todo(fd, &fin, 1);
48 }
```

recibir_linea lee el comando que envía el cliente. Se lee un byte a la vez hasta encontrar el salto de línea, para no tomar por error el inicio del siguiente comando. La función regresa el número de bytes leídos, cero si el cliente cerró la conexión y menos uno si hubo un error.

```
50 /* Lee del socket caracter por caracter hasta el salto de linea.
51    Regresa cuantos bytes se leyeron, 0 si el cliente cerro o -1 si hubo error */
52 static int recibir_linea(int fd, char *buf, size_t tam)
53 {
54     size_t n = 0;
55     int leidos = 0;
56     char c;
57     ssize_t r;
58 
59     while (n < tam - 1) {
60         r = recv(fd, &c, 1, 0);
61         if (r < 0)
62             return -1;
63         if (r == 0)
64             break;
65         leidos++;
66         if (c == '\n')
67             break;
68         buf[n++] = c;
69     }
70     buf[n] = '\0';
71     return leidos;
72 }
```

atender_cliente es la conversación completa con un cliente y se ejecuta dentro del proceso hijo. En cada vuelta del ciclo se recibe un comando, se imprime en la terminal del servidor junto con la IP del cliente, y se revisa si es SALIR. Si lo es, se envía una despedida y termina el ciclo. Si no, el comando se pasa a **sql_procesar**, que arma la respuesta, y ésta se envía al cliente.

```
74 /* Ciclo de atencion de un cliente: recibe un comando, lo ejecuta y responde,
75    hasta que llega SALIR o el cliente cierra la conexion */
76 static void atender_cliente(int fd_c, const char *ip_cliente, int num_cliente)
77 {
78     char peticion[TAM_PETICION];
79     char *respuesta = malloc(TAM_RESPUESTA);
80     int n;
81 
82     if (respuesta == NULL) {
83         perror("malloc");
84         return;
```

```
 85      }
 86
 87      while (1) {
 88          n = recibir_linea(fd_c, peticion, sizeof(peticion));
 89          if (n <= 0) {
 90              printf("[cliente %d - %s] cerro la conexion\n", num_cliente, ip_cliente);
 91              break;
 92          }
 93
 94          recortar_cadena(peticion);
 95          printf("[cliente %d - %s] comando: %s\n", num_cliente, ip_cliente, peticion);
 96
 97          if (strcasecmp(peticion, CMD_SALIR) == 0) {
 98              enviar_respuesta(fd_c, "Adios. Conexion cerrada por el servidor.\n");
 99              break;
100          }
101
102          sql_procesar(peticion, respuesta, TAM_RESPUESTA);
103
104          if (enviar_respuesta(fd_c, respuesta) != 0) {
105              printf("[cliente %d - %s] error al enviar la respuesta\n", num_cliente,
ip_cliente);
106              break;
107          }
108      }
109
110      free(respuesta);
111  }
```

La función principal sigue la secuencia típica de un servidor TCP. Primero se crea el socket con `socket` y se activa la opción `SO_REUSEADDR` para poder reiniciar el servidor sin esperar a que el puerto se libere. Después se llena la estructura `sockaddr_in` con la familia AF_INET, la dirección `INADDR_ANY` para aceptar conexiones por cualquier interfaz y el puerto convertido a orden de red con `htons`. Con `bind` se asocia el socket a esa dirección y con `listen` se pone a escuchar.

La llamada a `signal` con `SIGCHLD` hace que el sistema operativo recoja de manera automática a los procesos hijos cuando terminan, para que no queden procesos zombie.

En el ciclo principal, `accept` se bloquea hasta que llega un cliente y regresa un nuevo descriptor para hablar con él. Se imprime la fecha, la hora y la IP del cliente, que se obtiene con `inet_ntoa`. Enseguida se llama a `fork`: el proceso hijo cierra el socket de escucha, atiende al cliente y termina; el proceso padre cierra el descriptor del cliente y regresa a `accept` para esperar al siguiente. Así, cada cliente tiene su propio proceso y varios pueden estar conectados al mismo tiempo.

```
113  int main(int argc, char *argv[])
114  {
115      struct sockaddr_in servidor;
116      struct sockaddr_in cliente;
117      socklen_t long_cliente;
118      int fd_s, fd_c;
```

```
119     int puerto = PUERTO_DEFAULT;
120     int num_cliente = 0;
121     int opcion = 1;
122 
123     if (argc >= 2)
124         puerto = atoi(argv[1]);
125     if (puerto <= 0 || puerto > 65535) {
126         fprintf(stderr, "Uso: %s [puerto]\n", argv[0]);
127         exit(1);
128     }
129 
130     /* Se crea el socket TCP y se permite reutilizar el puerto al reiniciar */
131     fd_s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
132     if (fd_s < 0) {
133         perror("socket");
134         exit(1);
135     }
136     if (setsockopt(fd_s, SOL_SOCKET, SO_REUSEADDR, &opcion, sizeof(opcion)) < 0) {
137         perror("setsockopt");
138         exit(1);
139     }
140 
141     /* Se asocia el socket al puerto en todas las interfaces y se pone a escuchar */
142     memset((char *) &servidor, 0, sizeof(servidor));
143     servidor.sin_family = AF_INET;
144     servidor.sin_addr.s_addr = INADDR_ANY;
145     servidor.sin_port = htons((u_short) puerto);
146 
147     if (bind(fd_s, (struct sockaddr *) &servidor, sizeof(servidor)) < 0) {
148         perror("bind");
149         exit(1);
150     }
151     if (listen(fd_s, QLEN) < 0) {
152         perror("listen");
153         exit(1);
154     }
155 
156     /* Los hijos que terminan se limpian solos */
157     signal(SIGCHLD, SIG_IGN);
158 
159     printf("Servidor escuchando en el puerto %d. Ctrl-C para terminar.\n", puerto);
160 
161     /* Por cada cliente aceptado se crea un hijo que lo atiende */
162     while (1) {
163         long_cliente = sizeof(cliente);
164         fd_c = accept(fd_s, (struct sockaddr *) &cliente, &long_cliente);
165         if (fd_c < 0) {
166             perror("accept");
167             continue;
168         }
169         num_cliente++;
170 
171         time_t T = time(NULL);
172         struct tm tm = *localtime(&T);
173         printf("%02d/%02d/%04d %02d:%02d:%02d - Cliente %d conectado desde %s:%d\n",
174                tm.tm_mday, tm.tm_mon + 1, tm.tm_year + 1900,
175                tm.tm_hour, tm.tm_min, tm.tm_sec,
176                num_cliente, inet_ntoa(cliente.sin_addr), ntohs(cliente.sin_port));
177 
```

```
178         if (fork() == 0) {
179             close(fd_s);
180             atender_cliente(fd_c, inet_ntoa(cliente.sin_addr), num_cliente);
181             close(fd_c);
182             printf("Cliente %d desconectado.\n", num_cliente);
183             exit(0);
184         } else {
185             close(fd_c);
186         }
187     }
188 
189     close(fd_s);
190     return 0;
191 }
```

Cliente

El cliente se encuentra en **`cliente.c`**. Recibe como argumentos la dirección IP o el nombre del servidor y, de forma opcional, el puerto.

```
 1 /*
 2  * Cliente del manejador de bases de datos.
 3  * Se conecta al servidor por TCP, envia los comandos SQL que escribe
 4  * el usuario y muestra la respuesta. Termina con el comando SALIR.
 5  *
 6  * Uso: ./bin/cliente <ip_servidor> [puerto]
 7  */
 8 #include <stdio.h>
 9 #include <stdlib.h>
10 #include <string.h>
11 #include <strings.h>
12 #include <unistd.h>
13 #include <sys/types.h>
14 #include <sys/socket.h>
15 #include <netinet/in.h>
16 #include <arpa/inet.h>
17 #include <netdb.h>
18 
19 #include "protocolo.h"
20 #include "util.h"
```

La función **enviar_todo** es la misma que en el servidor. **recibir_respuesta** lee del socket en un ciclo e imprime en pantalla lo que va llegando. En cada bloque recibido busca el carácter de fin de respuesta; si lo encuentra, imprime lo que había antes de él y termina. Si `recv` regresa cero es que el servidor cerró la conexión.

```
22 /* Envia el buffer completo repitiendo send hasta terminar */
23 static int enviar_todo(int sd, const char *datos, size_t cantidad)
24 {
25     size_t enviados = 0;
26     ssize_t r;
27 
28     while (enviados < cantidad) {
29         r = send(sd, datos + enviados, cantidad - enviados, 0);
```

```
30          if (r <= 0)
31              return -1;
32          enviados += (size_t) r;
33      }
34      return 0;
35  }
36
37  /* Lee la respuesta del servidor y la imprime conforme llega,
38     hasta encontrar el caracter de fin de respuesta */
39  static int recibir_respuesta(int sd)
40  {
41      char buf[TAM_RESPUESTA];
42      ssize_t n;
43      ssize_t i;
44
45      while (1) {
46          n = recv(sd, buf, sizeof(buf), 0);
47          if (n <= 0)
48              return -1;
49
50          for (i = 0; i < n; i++) {
51              if (buf[i] == FIN_RESPUESTA)
52                  break;
53          }
54
55          if (i > 0)
56              write(1, buf, (size_t) i);
57
58          if (i < n)
59              return 0;
60      }
61  }
```

En la función principal se crea el socket, se obtiene la dirección del servidor con `gethostbyname`, que acepta tanto una IP como un nombre, y se copia a la estructura `sockaddr_in` junto con el puerto en orden de red. Con `connect` se establece la conexión.

Después viene el ciclo de la conversación. El cliente muestra el indicador `SQL>`, lee una línea del teclado, le quita los espacios sobrantes y la envía al servidor con un salto de línea al final. Enseguida espera la respuesta completa y la muestra. Si el comando fue SALIR, después de recibir la despedida se cierra el socket y el programa termina. Las líneas vacías no se envían y la combinación Ctrl-D se trata como SALIR para no dejar la conexión abierta.

```
63  int main(int argc, char *argv[])
64  {
65      struct sockaddr_in server;
66      struct hostent *sp;
67      int sd;
68      int puerto = PUERTO_DEFAULT;
69      char linea[TAM_PETICION];
70      char peticion[TAM_PETICION + 1];   /* comando mas el salto de linea */
71      int es_salir;
72
```

```
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <ip_servidor> [puerto]\n", argv[0]);
        exit(1);
    }
    if (argc >= 3)
        puerto = atoi(argv[2]);
    if (puerto <= 0 || puerto > 65535) {
        fprintf(stderr, "Puerto invalido: %s\n", argv[2]);
        exit(1);
    }

    /* Se crea el socket y se obtiene la direccion del servidor */
    sd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sd < 0) {
        perror("socket");
        exit(1);
    }

    sp = gethostbyname(argv[1]);
    if (sp == NULL) {
        fprintf(stderr, "No se pudo resolver el host: %s\n", argv[1]);
        exit(1);
    }

    memset((char *) &server, 0, sizeof(struct sockaddr_in));
    server.sin_family = AF_INET;
    server.sin_port = htons((u_short) puerto);
    memcpy(&server.sin_addr, sp->h_addr, sp->h_length);

    /* Conexion con el servidor */
    if (connect(sd, (struct sockaddr *) &server, sizeof(struct sockaddr_in)) < 0) {
        perror("connect");
        exit(1);
    }

    printf("Conectado al servidor %s:%d\n", inet_ntoa(server.sin_addr), puerto);
    printf("Escribe un comando SQL por linea. SALIR para terminar.\n");

    /* Se lee un comando del teclado, se envia y se espera la respuesta */
    while (1) {
        printf("SQL> ");
        fflush(stdout);

        if (fgets(linea, sizeof(linea), stdin) == NULL) {
            printf("\n");
            strcpy(linea, CMD_SALIR);
        }

        recortar_cadena(linea);
        if (linea[0] == '\0')
            continue;

        es_salir = (strcasecmp(linea, CMD_SALIR) == 0);

        snprintf(peticion, sizeof(peticion), "%s\n", linea);
        if (enviar_todo(sd, peticion, strlen(peticion)) != 0) {
            printf("No se pudo enviar el comando. El servidor cerro la conexion.\n");
            break;
        }
```

```
132
133         if (recibir_respuesta(sd) != 0) {
134             printf("El servidor cerro la conexion.\n");
135             break;
136         }
137
138         if (es_salir)
139             break;
140     }
141
142     close(sd);
143     return 0;
144 }
```

Manejo de la tabla

El archivo **`tabla.h`** define cómo se representa una tabla en memoria. Un **Registro** es un arreglo de campos de texto y una **Tabla** guarda el nombre, la ruta del archivo, los nombres de las columnas y un arreglo de registros. También se declaran las constantes con los límites del proyecto: hasta 8 columnas, 1000 registros y 64 caracteres por campo.

```
 1 /*
 2  * Manejo de una tabla guardada en un archivo de texto.
 3  * La primera linea del archivo es la cabecera y cada linea siguiente es
 4  * un registro. Los campos van separados por coma. La primera columna es
 5  * el Id y se genera de forma automatica.
 6  */
 7 #ifndef TABLA_H
 8 #define TABLA_H
 9
10 #include <stdio.h>
11
12 #define DIRECTORIO_DATOS  "datos"
13 #define EXTENSION_TABLA   ".txt"
14 #define SEPARADOR         ','
15 #define MAX_COLUMNAS      8
16 #define MAX_REGISTROS     1000
17 #define MAX_VALOR         64
18 #define MAX_LINEA         1024
19 #define MAX_RUTA          128
20 #define TAM_ERROR         256
21
22 typedef struct {
23     char campos[MAX_COLUMNAS][MAX_VALOR];
24 } Registro;
25
26 typedef struct {
27     char nombre[MAX_VALOR];
28     char ruta[MAX_RUTA];
29     FILE *archivo;
30     int escritura;
31     int num_columnas;
32     char columnas[MAX_COLUMNAS][MAX_VALOR];
33     int num_registros;
34     Registro *registros;
35 } Tabla;
```

```
36
37 /* Abre el archivo de la tabla y carga la cabecera y los registros en memoria */
38 int tabla_cargar(const char *nombre, int escritura, Tabla *t, char *error);
39
40 /* Escribe la tabla completa de regreso al archivo */
41 int tabla_guardar(Tabla *t, char *error);
42
43 /* Cierra el archivo y libera la memoria */
44 void tabla_liberar(Tabla *t);
45
46 /* Regresa la posicion de una columna por su nombre, o -1 si no existe */
47 int tabla_indice_columna(const Tabla *t, const char *columna);
48
49 /* Regresa el Id que le toca al siguiente registro */
50 int tabla_siguiente_id(const Tabla *t);
51
52 /* Separa una linea por comas y guarda cada parte en campos */
53 int tabla_dividir_campos(char *linea, char campos[][MAX_VALOR]);
54
55 #endif
```

En **`tabla.c`** está la lógica para pasar del archivo a la memoria y de regreso. La idea general es cargar la tabla completa en memoria, trabajar ahí y, si hubo cambios, reescribir el archivo entero. Como el servidor crea un proceso por cliente, dos procesos podrían escribir el archivo al mismo tiempo; para evitarlo, mientras una tabla está cargada su archivo se bloquea con `flock`: con bloqueo compartido si solo se va a leer y con bloqueo exclusivo si se va a modificar.

tabla_dividir_campos recorre una línea buscando comas y copia cada pedazo como un campo, quitándole los espacios. Se usa para la cabecera, para cada registro y también para la lista de valores del INSERT. **nombre_valido** revisa que el nombre de la tabla solo tenga letras, dígitos y guion bajo, de modo que no se pueda abrir un archivo fuera de la carpeta de datos.

```
1  /*
2   * Manejo de una tabla guardada en un archivo de texto.
3   * La tabla se carga completa en memoria, se modifica ahi y al guardar
4   * se reescribe el archivo. Mientras esta cargada, el archivo queda
5   * bloqueado para que dos procesos no lo escriban al mismo tiempo.
6   */
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <string.h>
10 #include <strings.h>
11 #include <ctype.h>
12 #include <unistd.h>
13 #include <sys/file.h>
14
15 #include "tabla.h"
16 #include "util.h"
17
18 /* Recorre la linea buscando comas y copia cada pedazo como un campo */
19 int tabla_dividir_campos(char *linea, char campos[][MAX_VALOR])
```

```c
{
    int n = 0;
    char *token = linea;
    char *coma;

    while (n < MAX_COLUMNAS) {
        coma = strchr(token, SEPARADOR);
        if (coma != NULL)
            *coma = '\0';
        strncpy(campos[n], token, MAX_VALOR - 1);
        campos[n][MAX_VALOR - 1] = '\0';
        recortar_cadena(campos[n]);
        n++;
        if (coma == NULL)
            break;
        token = coma + 1;
    }
    return n;
}

/* El nombre de la tabla solo puede tener letras, digitos y guion bajo */
static int nombre_valido(const char *nombre)
{
    size_t i;

    if (nombre[0] == '\0')
        return 0;
    for (i = 0; nombre[i] != '\0'; i++) {
        if (!isalnum((unsigned char) nombre[i]) && nombre[i] != '_')
            return 0;
    }
    return 1;
}
```

tabla_cargar forma la ruta del archivo con el nombre de la tabla en minúsculas, por ejemplo `Alumno` se convierte en `datos/alumno.txt`. Abre el archivo en modo lectura o en modo lectura y escritura según se pida, lo bloquea, reserva memoria para los registros y lee la primera línea como cabecera. Después lee cada línea siguiente como un registro, ignorando las líneas vacías. Si algo falla, deja un mensaje en el parámetro `error` y regresa menos uno.

```c
/* Forma la ruta del archivo con el nombre en minusculas, lo abre, lo bloquea
   y lee la cabecera y despues cada registro */
int tabla_cargar(const char *nombre, int escritura, Tabla *t, char *error)
{
    char linea[MAX_LINEA];
    char nombre_min[MAX_VALOR];
    size_t i;

    memset(t, 0, sizeof(Tabla));
    strncpy(t->nombre, nombre, MAX_VALOR - 1);
    t->escritura = escritura;

    if (!nombre_valido(nombre)) {
        snprintf(error, TAM_ERROR, "ERROR: nombre de tabla invalido '%s'", nombre);
        return -1;
```

```
 69      }
 70
 71      for (i = 0; nombre[i] != '\0' && i < MAX_VALOR - 1; i++)
 72          nombre_min[i] = (char) tolower((unsigned char) nombre[i]);
 73      nombre_min[i] = '\0';
 74      snprintf(t->ruta, sizeof(t->ruta), "%s/%s%s", DIRECTORIO_DATOS, nombre_min,
EXTENSION_TABLA);
 75
 76      t->archivo = fopen(t->ruta, escritura ? "r+" : "r");
 77      if (t->archivo == NULL) {
 78          snprintf(error, TAM_ERROR, "ERROR: la tabla '%s' no existe, no se encontro el
archivo %s",
 79                   nombre, t->ruta);
 80          return -1;
 81      }
 82
 83      if (flock(fileno(t->archivo), escritura ? LOCK_EX : LOCK_SH) != 0) {
 84          snprintf(error, TAM_ERROR, "ERROR: no se pudo bloquear el archivo %s", t->ruta);
 85          tabla_liberar(t);
 86          return -1;
 87      }
 88
 89      t->registros = calloc(MAX_REGISTROS, sizeof(Registro));
 90      if (t->registros == NULL) {
 91          snprintf(error, TAM_ERROR, "ERROR: sin memoria para cargar la tabla");
 92          tabla_liberar(t);
 93          return -1;
 94      }
 95
 96      if (fgets(linea, sizeof(linea), t->archivo) == NULL) {
 97          snprintf(error, TAM_ERROR, "ERROR: el archivo %s esta vacio, no tiene cabecera", t-
>ruta);
 98          tabla_liberar(t);
 99          return -1;
100      }
101      t->num_columnas = tabla_dividir_campos(linea, t->columnas);
102
103      while (fgets(linea, sizeof(linea), t->archivo) != NULL) {
104          recortar_cadena(linea);
105          if (linea[0] == '\0')
106              continue;
107          if (t->num_registros >= MAX_REGISTROS) {
108              snprintf(error, TAM_ERROR, "ERROR: la tabla excede el maximo de %d registros",
MAX_REGISTROS);
109              tabla_liberar(t);
110              return -1;
111          }
112          tabla_dividir_campos(linea, t->registros[t->num_registros].campos);
113          t->num_registros++;
114      }
115
116      return 0;
117  }
```

tabla_guardar vacía el archivo con `ftruncate` y vuelve a escribir la cabecera y todos los registros, separando los campos con coma. **tabla_liberar** quita el bloqueo, cierra el archivo y libera la memoria. **tabla_indice_columna** busca una columna por nombre sin distinguir

mayúsculas y **tabla_siguiente_id** busca el Id más grande y le suma uno, que es el valor que recibe el siguiente registro insertado.

```
119 /* Vacia el archivo y vuelve a escribir la cabecera y todos los registros */
120 int tabla_guardar(Tabla *t, char *error)
121 {
122     int i, j;
123 
124     if (!t->escritura || t->archivo == NULL) {
125         snprintf(error, TAM_ERROR, "ERROR: la tabla no se abrio para escritura");
126         return -1;
127     }
128 
129     rewind(t->archivo);
130     if (ftruncate(fileno(t->archivo), 0) != 0) {
131         snprintf(error, TAM_ERROR, "ERROR: no se pudo escribir el archivo %s", t->ruta);
132         return -1;
133     }
134 
135     for (j = 0; j < t->num_columnas; j++)
136         fprintf(t->archivo, "%s%c", t->columnas[j],
137                 (j < t->num_columnas - 1) ? SEPARADOR : '\n');
138 
139     for (i = 0; i < t->num_registros; i++) {
140         for (j = 0; j < t->num_columnas; j++)
141             fprintf(t->archivo, "%s%c", t->registros[i].campos[j],
142                     (j < t->num_columnas - 1) ? SEPARADOR : '\n');
143     }
144 
145     if (fflush(t->archivo) != 0) {
146         snprintf(error, TAM_ERROR, "ERROR: no se pudo escribir el archivo %s", t->ruta);
147         return -1;
148     }
149     return 0;
150 }
151 
152 /* Quita el bloqueo, cierra el archivo y libera los registros */
153 void tabla_liberar(Tabla *t)
154 {
155     if (t->archivo != NULL) {
156         fflush(t->archivo);
157         flock(fileno(t->archivo), LOCK_UN);
158         fclose(t->archivo);
159         t->archivo = NULL;
160     }
161     free(t->registros);
162     t->registros = NULL;
163 }
164 
165 /* Compara el nombre con cada columna sin distinguir mayusculas */
166 int tabla_indice_columna(const Tabla *t, const char *columna)
167 {
168     int j;
169 
170     for (j = 0; j < t->num_columnas; j++) {
171         if (strcasecmp(t->columnas[j], columna) == 0)
172             return j;
173     }
```

```
    return -1;
}

/* Busca el Id mas grande y le suma uno */
int tabla_siguiente_id(const Tabla *t)
{
    int i, id, mayor = 0;

    for (i = 0; i < t->num_registros; i++) {
        id = atoi(t->registros[i].campos[0]);
        if (id > mayor)
            mayor = id;
    }
    return mayor + 1;
}
```

Intérprete de comandos SQL

El archivo **`sql.c`** contiene una función por cada comando y una función de entrada que decide a cuál llamar. Todas reciben el comando y el buffer donde se arma la respuesta. La función **agregar** escribe texto con formato al final de la respuesta cuidando no pasarse del tamaño del buffer.

```
/* Interprete de los comandos SQL que recibe el servidor */
#ifndef SQL_H
#define SQL_H

#include <stddef.h>

/* Ejecuta el comando y deja en respuesta el resultado o un mensaje de error */
void sql_procesar(const char *comando, char *respuesta, size_t tam);

#endif
```

```
/*
 * Interprete de los comandos SQL que recibe el servidor.
 * Cada comando se revisa con funciones de cadenas, se ejecuta sobre la
 * tabla y se arma el texto de respuesta para el cliente.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <stdarg.h>
#include <ctype.h>

#include "protocolo.h"
#include "sql.h"
#include "tabla.h"
#include "util.h"

/* Agrega texto con formato al final de la respuesta sin pasarse del tamanio */
static void agregar(char *respuesta, size_t tam, const char *formato, ...)
{
    size_t usado = strlen(respuesta);
```

```
    va_list args;

    if (usado >= tam - 1)
        return;
    va_start(args, formato);
    vsnprintf(respuesta + usado, tam - usado, formato, args);
    va_end(args);
}
```

Las siguientes funciones son de apoyo para analizar los comandos. **buscar_palabra** localiza una palabra clave como VALUES, SET o WHERE sin distinguir mayúsculas y verificando que sea una palabra completa. **copiar_entre** copia el pedazo de comando que hay entre dos posiciones, por ejemplo lo que está entre SET y WHERE. **separar_asignacion** parte una expresión de la forma columna=valor en sus dos partes. **cumple_condicion** compara el campo de un registro con el valor de la condición WHERE.

```
/* Busca una palabra clave dentro del comando sin distinguir mayusculas.
   Solo cuenta si esta separada del resto por espacios o parentesis */
static const char *buscar_palabra(const char *texto, const char *palabra)
{
    size_t len = strlen(palabra);
    const char *p;

    for (p = texto; *p != '\0'; p++) {
        if (strncasecmp(p, palabra, len) != 0)
            continue;
        if (p != texto && !isspace((unsigned char) p[-1]) && p[-1] != ')')
            continue;
        if (p[len] != '\0' && !isspace((unsigned char) p[len]) && p[len] != '(')
            continue;
        return p;
    }
    return NULL;
}

/* Copia el texto que hay entre dos posiciones y le quita los espacios */
static void copiar_entre(const char *inicio, const char *fin, char *destino, size_t tam)
{
    size_t len = (fin > inicio) ? (size_t) (fin - inicio) : 0;

    if (len >= tam)
        len = tam - 1;
    memcpy(destino, inicio, len);
    destino[len] = '\0';
    recortar_cadena(destino);
}

/* Separa una expresion columna=valor en sus dos partes */
static int separar_asignacion(char *expresion, char *columna, char *valor)
{
    char *igual = strchr(expresion, '=');

    if (igual == NULL)
        return -1;
    *igual = '\0';

```

```
    strncpy(columna, expresion, MAX_VALOR - 1);
    columna[MAX_VALOR - 1] = '\0';
    recortar_cadena(columna);

    strncpy(valor, igual + 1, MAX_VALOR - 1);
    valor[MAX_VALOR - 1] = '\0';
    recortar_cadena(valor);
    quitar_comillas(valor);

    return (columna[0] == '\0') ? -1 : 0;
}

/* Revisa si el campo del registro es igual al valor del WHERE */
static int cumple_condicion(const Tabla *t, int fila, int idx, const char *valor)
{
    return strcasecmp(t->registros[fila].campos[idx], valor) == 0;
}
```

Comando SELECT

sql_select espera exactamente cuatro palabras: SELECT, un asterisco, FROM y el nombre de la tabla. Carga la tabla en modo lectura y copia a la respuesta la cabecera y después cada registro, con el mismo formato que tiene el archivo. Por eso el cliente muestra el listado tal como está en `alumno.txt`.

```
/* SELECT * FROM tabla
   Carga la tabla y copia la cabecera y todos los registros a la respuesta */
static void sql_select(const char *cmd, char *resp, size_t tam)
{
    char k1[16], k2[16], k3[16], tabla[MAX_VALOR], extra[16];
    char error[TAM_ERROR];
    Tabla t;
    int i, j, n;

    n = sscanf(cmd, "%15s %15s %15s %63s %15s", k1, k2, k3, tabla, extra);
    if (n != 4 || strcmp(k2, "*") != 0 || strcasecmp(k3, "FROM") != 0) {
        agregar(resp, tam, "ERROR de sintaxis. Uso: SELECT * FROM <tabla>\n");
        return;
    }

    if (tabla_cargar(tabla, 0, &t, error) != 0) {
        agregar(resp, tam, "%s\n", error);
        return;
    }

    for (j = 0; j < t.num_columnas; j++)
        agregar(resp, tam, "%s%c", t.columnas[j],
                (j < t.num_columnas - 1) ? SEPARADOR : '\n');

    for (i = 0; i < t.num_registros; i++) {
        for (j = 0; j < t.num_columnas; j++)
            agregar(resp, tam, "%s%c", t.registros[i].campos[j],
                    (j < t.num_columnas - 1) ? SEPARADOR : '\n');
    }

```

```
119      tabla_liberar(&t);
120  }
```

Comando INSERT

sql_insert revisa que el comando empiece con INSERT INTO y el nombre de la tabla, localiza la palabra VALUES y toma el texto que hay entre el paréntesis que abre y el que cierra. Ese texto se separa por comas y a cada valor se le quitan las comillas si las tiene. Después se carga la tabla en modo escritura y se verifica que el número de valores sea igual al número de columnas menos uno, porque el Id no se envía. Se forma el registro nuevo con el Id que regresa **tabla_siguiente_id**, se agrega al final y se guarda el archivo. La respuesta indica el Id que recibió el registro.

```
122  /* INSERT INTO tabla VALUES valor1,valor2,...
123     Toma los valores entre parentesis, les asigna el siguiente Id
124     y agrega el registro al final de la tabla */
125  static void sql_insert(const char *cmd, char *resp, size_t tam)
126  {
127      char k1[16], k2[16], tabla[MAX_VALOR];
128      char valores[MAX_LINEA];
129      char campos[MAX_COLUMNAS][MAX_VALOR];
130      char error[TAM_ERROR];
131      const char *values, *abre, *cierra, *p;
132      size_t len;
133      Tabla t;
134      Registro nuevo;
135      int n, j, nuevo_id;
136
137      if (sscanf(cmd, "%15s %15s %63s", k1, k2, tabla) != 3 || strcasecmp(k2, "INTO") != 0) {
138          agregar(resp, tam, "ERROR de sintaxis. Uso: INSERT INTO <tabla> VALUES
(v1,v2,...)\n");
139          return;
140      }
141
142      values = buscar_palabra(cmd, "VALUES");
143      abre = (values != NULL) ? strchr(values, '(') : NULL;
144      cierra = (values != NULL) ? strrchr(values, ')') : NULL;
145      if (abre == NULL || cierra == NULL || cierra < abre) {
146          agregar(resp, tam, "ERROR de sintaxis. Uso: INSERT INTO <tabla> VALUES
(v1,v2,...)\n");
147          return;
148      }
149      for (p = cierra + 1; *p != '\0'; p++) {
150          if (!isspace((unsigned char) *p)) {
151              agregar(resp, tam, "ERROR de sintaxis: texto inesperado despues de ')'\n");
152              return;
153          }
154      }
155
156      len = (size_t) (cierra - abre - 1);
157      if (len >= sizeof(valores)) {
158          agregar(resp, tam, "ERROR: la lista de valores es demasiado larga\n");
159          return;
160      }
161      memcpy(valores, abre + 1, len);
```

```
    valores[len] = '\0';

    n = tabla_dividir_campos(valores, campos);
    for (j = 0; j < n; j++)
        quitar_comillas(campos[j]);

    if (tabla_cargar(tabla, 1, &t, error) != 0) {
        agregar(resp, tam, "%s\n", error);
        return;
    }

    if (n != t.num_columnas - 1) {
        agregar(resp, tam, "ERROR: se esperaban %d valores (", t.num_columnas - 1);
        for (j = 1; j < t.num_columnas; j++)
            agregar(resp, tam, "%s%s", t.columnas[j], (j < t.num_columnas - 1) ? "," : "");
        agregar(resp, tam, ") y se recibieron %d\n", n);
        tabla_liberar(&t);
        return;
    }
    for (j = 0; j < n; j++) {
        if (campos[j][0] == '\0') {
            agregar(resp, tam, "ERROR: el valor de la columna %s esta vacio\n",
t.columnas[j + 1]);
            tabla_liberar(&t);
            return;
        }
    }
    if (t.num_registros >= MAX_REGISTROS) {
        agregar(resp, tam, "ERROR: la tabla esta llena (%d registros)\n", MAX_REGISTROS);
        tabla_liberar(&t);
        return;
    }

    memset(&nuevo, 0, sizeof(nuevo));
    nuevo_id = tabla_siguiente_id(&t);
    snprintf(nuevo.campos[0], MAX_VALOR, "%d", nuevo_id);
    for (j = 0; j < n; j++)
        strcpy(nuevo.campos[j + 1], campos[j]);

    t.registros[t.num_registros] = nuevo;
    t.num_registros++;

    if (tabla_guardar(&t, error) != 0) {
        agregar(resp, tam, "%s\n", error);
        tabla_liberar(&t);
        return;
    }

    agregar(resp, tam, "OK: registro insertado en %s con Id=%d\n", t.nombre, nuevo_id);
    tabla_liberar(&t);
}
```

Comando UPDATE

sql_update localiza las palabras SET y WHERE; las dos son obligatorias, para no modificar toda la tabla por accidente. El texto entre SET y WHERE son las asignaciones, que pueden ser varias separadas por coma, y el texto después de WHERE es la condición. Se carga la

tabla, se verifica que todas las columnas existan y se rechaza el comando si alguna asignación intenta cambiar la primera columna, que es el Id. Después se recorren los registros y a los que cumplen la condición se les copian los valores nuevos. Si ningún registro cumplió, se responde con un error y el archivo no se toca; si hubo cambios, se guarda y se responde cuántos registros se actualizaron.

```
213  /* UPDATE tabla SET columna=valor WHERE columna=valor
214     Cambia los valores indicados en los registros que cumplen la condicion.
215     La columna Id no se puede modificar */
216  static void sql_update(const char *cmd, char *resp, size_t tam)
217  {
218      char k1[16], tabla[MAX_VALOR];
219      char asignaciones[MAX_LINEA], condicion[MAX_LINEA];
220      char col_where[MAX_VALOR], val_where[MAX_VALOR];
221      char cols[MAX_COLUMNAS][MAX_VALOR], vals[MAX_COLUMNAS][MAX_VALOR];
222      int idx[MAX_COLUMNAS];
223      char error[TAM_ERROR];
224      const char *set, *where;
225      char *token, *coma;
226      Tabla t;
227      int num_asig = 0, idx_where, afectados = 0, i, j;
228
229      if (sscanf(cmd, "%15s %63s", k1, tabla) != 2) {
230          agregar(resp, tam, "ERROR de sintaxis. Uso: UPDATE <tabla> SET <columna>=<valor>
WHERE <columna>=<valor>\n");
231          return;
232      }
233
234      set = buscar_palabra(cmd, "SET");
235      where = buscar_palabra(cmd, "WHERE");
236      if (set == NULL || where == NULL || where < set) {
237          agregar(resp, tam, "ERROR de sintaxis. Uso: UPDATE <tabla> SET <columna>=<valor>
WHERE <columna>=<valor>\n");
238          return;
239      }
240
241      copiar_entre(set + strlen("SET"), where, asignaciones, sizeof(asignaciones));
242      copiar_entre(where + strlen("WHERE"), cmd + strlen(cmd), condicion, sizeof(condicion));
243
244      if (separar_asignacion(condicion, col_where, val_where) != 0) {
245          agregar(resp, tam, "ERROR de sintaxis en WHERE. Uso: WHERE <columna>=<valor>\n");
246          return;
247      }
248
249      token = asignaciones;
250      while (token != NULL) {
251          if (num_asig >= MAX_COLUMNAS) {
252              agregar(resp, tam, "ERROR: demasiadas asignaciones en SET\n");
253              return;
254          }
255          coma = strchr(token, ',');
256          if (coma != NULL)
257              *coma = '\0';
258          if (separar_asignacion(token, cols[num_asig], vals[num_asig]) != 0) {
259              agregar(resp, tam, "ERROR de sintaxis en SET. Uso: SET <columna>=<valor>[,
<columna>=<valor>]\n");
```

```
260             return;
261         }
262         num_asig++;
263         token = (coma != NULL) ? coma + 1 : NULL;
264     }
265
266     if (tabla_cargar(tabla, 1, &t, error) != 0) {
267         agregar(resp, tam, "%s\n", error);
268         return;
269     }
270
271     idx_where = tabla_indice_columna(&t, col_where);
272     if (idx_where < 0) {
273         agregar(resp, tam, "ERROR: la columna '%s' no existe en %s\n", col_where,
t.nombre);
274         tabla_liberar(&t);
275         return;
276     }
277     for (j = 0; j < num_asig; j++) {
278         idx[j] = tabla_indice_columna(&t, cols[j]);
279         if (idx[j] < 0) {
280             agregar(resp, tam, "ERROR: la columna '%s' no existe en %s\n", cols[j],
t.nombre);
281             tabla_liberar(&t);
282             return;
283         }
284         if (idx[j] == 0) {
285             agregar(resp, tam, "ERROR: la columna %s es la llave primaria y no puede
actualizarse\n",
286                     t.columnas[0]);
287             tabla_liberar(&t);
288             return;
289         }
290         if (vals[j][0] == '\0') {
291             agregar(resp, tam, "ERROR: el valor para la columna %s esta vacio\n",
t.columnas[idx[j]]);
292             tabla_liberar(&t);
293             return;
294         }
295     }
296
297     for (i = 0; i < t.num_registros; i++) {
298         if (!cumple_condicion(&t, i, idx_where, val_where))
299             continue;
300         for (j = 0; j < num_asig; j++)
301             strcpy(t.registros[i].campos[idx[j]], vals[j]);
302         afectados++;
303     }
304
305     if (afectados == 0) {
306         agregar(resp, tam, "ERROR: ningun registro de %s cumple la condicion %s=%s\n",
307                 t.nombre, col_where, val_where);
308         tabla_liberar(&t);
309         return;
310     }
311
312     if (tabla_guardar(&t, error) != 0) {
313         agregar(resp, tam, "%s\n", error);
314         tabla_liberar(&t);
```

```
315          return;
316      }
317
318      agregar(resp, tam, "OK: %d registro(s) actualizado(s) en %s\n", afectados, t.nombre);
319      tabla_liberar(&t);
320  }
```

Comando DELETE

sql_delete funciona de forma parecida. Localiza WHERE, obtiene la condición y carga la tabla. Para eliminar registros se recorre el arreglo con dos índices: los registros que no cumplen la condición se van copiando hacia adelante y los que sí cumplen se dejan fuera. Al final el número de registros se reduce y se guarda el archivo. Los Id de los registros restantes no se vuelven a numerar, porque el Id es la llave primaria y debe mantenerse estable.

```
322  /* DELETE FROM tabla WHERE columna=valor
323     Quita de la tabla los registros que cumplen la condicion */
324  static void sql_delete(const char *cmd, char *resp, size_t tam)
325  {
326      char k1[16], k2[16], tabla[MAX_VALOR];
327      char condicion[MAX_LINEA];
328      char col_where[MAX_VALOR], val_where[MAX_VALOR];
329      char error[TAM_ERROR];
330      const char *where;
331      Tabla t;
332      int idx_where, eliminados = 0, i, destino;
333
334      if (sscanf(cmd, "%15s %15s %63s", k1, k2, tabla) != 3 || strcasecmp(k2, "FROM") != 0) {
335          agregar(resp, tam, "ERROR de sintaxis. Uso: DELETE FROM <tabla> WHERE <columna>=
<valor>\n");
336          return;
337      }
338
339      where = buscar_palabra(cmd, "WHERE");
340      if (where == NULL) {
341          agregar(resp, tam, "ERROR de sintaxis. Uso: DELETE FROM <tabla> WHERE <columna>=
<valor>\n");
342          return;
343      }
344      copiar_entre(where + strlen("WHERE"), cmd + strlen(cmd), condicion, sizeof(condicion));
345      if (separar_asignacion(condicion, col_where, val_where) != 0) {
346          agregar(resp, tam, "ERROR de sintaxis en WHERE. Uso: WHERE <columna>=<valor>\n");
347          return;
348      }
349
350      if (tabla_cargar(tabla, 1, &t, error) != 0) {
351          agregar(resp, tam, "%s\n", error);
352          return;
353      }
354
355      idx_where = tabla_indice_columna(&t, col_where);
356      if (idx_where < 0) {
357          agregar(resp, tam, "ERROR: la columna '%s' no existe en %s\n", col_where,
t.nombre);
358          tabla_liberar(&t);
```

```
359         return;
360     }
361
362     /* Los registros que no cumplen se van recorriendo hacia adelante
363        y los que cumplen se dejan fuera */
364     destino = 0;
365     for (i = 0; i < t.num_registros; i++) {
366         if (cumple_condicion(&t, i, idx_where, val_where)) {
367             eliminados++;
368             continue;
369         }
370         if (destino != i)
371             t.registros[destino] = t.registros[i];
372         destino++;
373     }
374     t.num_registros = destino;
375
376     if (eliminados == 0) {
377         agregar(resp, tam, "ERROR: ningun registro de %s cumple la condicion %s=%s\n",
378                 t.nombre, col_where, val_where);
379         tabla_liberar(&t);
380         return;
381     }
382
383     if (tabla_guardar(&t, error) != 0) {
384         agregar(resp, tam, "%s\n", error);
385         tabla_liberar(&t);
386         return;
387     }
388
389     agregar(resp, tam, "OK: %d registro(s) eliminado(s) de %s\n", eliminados, t.nombre);
390     tabla_liberar(&t);
391 }
392
393 /* Limpia el comando, identifica la primera palabra y llama a la funcion
394    que le corresponde */
```

Función de entrada

sql_procesar es la función que llama el servidor. Hace una copia del comando, le quita los espacios y el punto y coma final si lo hay, y lee la primera palabra. Según esa palabra llama a la función del comando correspondiente; si no es ninguna de las conocidas, responde que el comando no se reconoce.

```
396 {
397     char cmd[TAM_PETICION];
398     char verbo[16];
399     size_t n;
400
401     respuesta[0] = '\0';
402
403     strncpy(cmd, comando, sizeof(cmd) - 1);
404     cmd[sizeof(cmd) - 1] = '\0';
405     recortar_cadena(cmd);
406     n = strlen(cmd);
407     if (n > 0 && cmd[n - 1] == ';') {
408         cmd[n - 1] = '\0';
```

```
409         recortar_cadena(cmd);
410     }
411
412     if (cmd[0] == '\0' || sscanf(cmd, "%15s", verbo) != 1) {
413         agregar(respuesta, tam, "ERROR: comando vacio\n");
414         return;
415     }
416
417     if (strcasecmp(verbo, "SELECT") == 0)
418         sql_select(cmd, respuesta, tam);
419     else if (strcasecmp(verbo, "INSERT") == 0)
420         sql_insert(cmd, respuesta, tam);
421     else if (strcasecmp(verbo, "UPDATE") == 0)
422         sql_update(cmd, respuesta, tam);
423     else if (strcasecmp(verbo, "DELETE") == 0)
424         sql_delete(cmd, respuesta, tam);
425     else
426         agregar(respuesta, tam,
427                 "ERROR: comando no reconocido '%s'. Comandos validos: SELECT, INSERT,
UPDATE, DELETE, SALIR\n",
428                 verbo);
429 }
```

Makefile

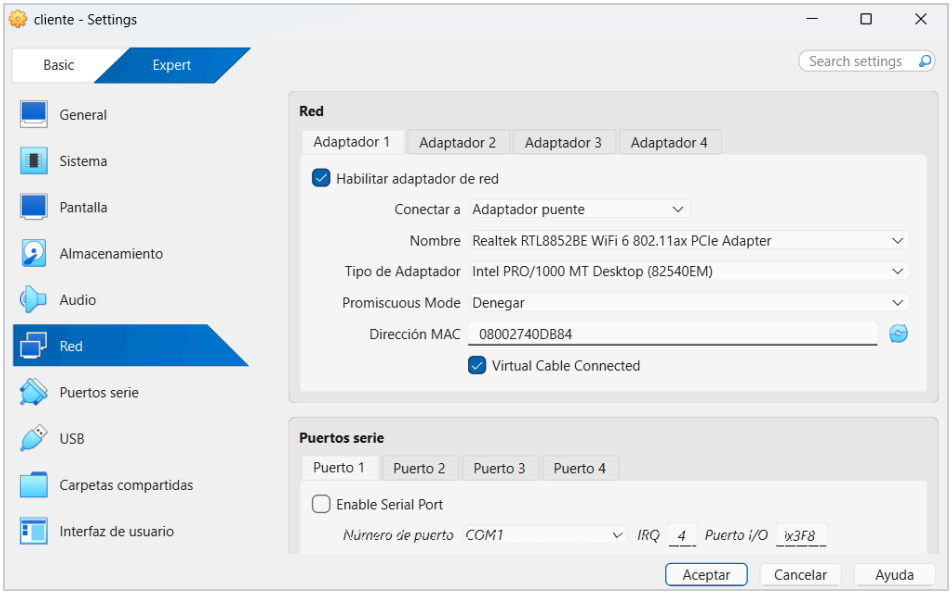
El **`Makefile`** compila los dos programas con las advertencias activadas. `make` genera `bin/servidor` y `bin/cliente`, y `make clean` borra los binarios.

```
1  CC     = gcc
2  CFLAGS = -Wall -Wextra -std=gnu99 -Isrc
3
4  SERVIDOR_SRC = src/servidor.c src/sql.c src/tabla.c src/util.c
5  CLIENTE_SRC  = src/cliente.c src/util.c
6
7  all: bin/servidor bin/cliente
8
9  bin:
10     mkdir -p bin
11
12 bin/servidor: $(SERVIDOR_SRC) src/protocolo.h src/sql.h src/tabla.h src/util.h | bin
13     $(CC) $(CFLAGS) -o $@ $(SERVIDOR_SRC)
14
15 bin/cliente: $(CLIENTE_SRC) src/protocolo.h src/util.h | bin
16     $(CC) $(CFLAGS) -o $@ $(CLIENTE_SRC)
17
18 clean:
19     rm -rf bin
20
21 .PHONY: all clean
```

RESULTADOS

Las pruebas se realizaron con dos máquinas virtuales de Ubuntu en VirtualBox, conectadas en modo de adaptador puente para que cada una tuviera su propia dirección IP en la red local. La máquina **servidor** recibió la dirección 192.168.100.102 y la máquina **cliente** la dirección 192.168.100.100.

Preparación de las máquinas



Configuración de red de la máquina virtual en modo adaptador puente.



Nombre y dirección IP de la máquina servidor.



Nombre y dirección IP de la máquina cliente.

```
ricardo@servidor:~$ ping -c 3 192.168.100.102
PING 192.168.100.102 (192.168.100.102) 56(84) bytes of data.
64 bytes from 192.168.100.102: icmp_seq=1 ttl=64 time=2.15 ms
64 bytes from 192.168.100.102: icmp_seq=2 ttl=64 time=1.22 ms
64 bytes from 192.168.100.102: icmp_seq=3 ttl=64 time=0.568 ms

--- 192.168.100.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 0.568/1.313/2.150/0.649 ms
ricardo@servidor:~$
```

Prueba de conectividad del cliente hacia el servidor con ping.

```
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ git pull
Already up to date.
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ make clean && make
rm -rf bin
mkdir -p bin
gcc -Wall -Wextra -std=gnu99 -Isrc -o bin/servidor src/servidor.c src/sql.c src/
tabla.c src/util.c
gcc -Wall -Wextra -std=gnu99 -Isrc -o bin/cliente src/cliente.c src/util.c
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$
```

Compilación del proyecto con make en una de las máquinas.

```
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ git pull
Already up to date.
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ make clean && make
rm -rf bin
mkdir -p bin
gcc -Wall -Wextra -std=gnu99 -Isrc -o bin/servidor src/servidor.c src/sql.c src/
tabla.c src/util.c
gcc -Wall -Wextra -std=gnu99 -Isrc -o bin/cliente src/cliente.c src/util.c
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ git checkout datos/al
umno.txt
Updated 1 path from the index
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$
```

Compilación en la otra máquina y restauración de la tabla original.

Conexión remota

```
ricardo@servidor: ~/Proyecto-Arquitectura-Cliente-S...
~/Proyecto-Arquitectura-Cliente-Servidor
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ ./bin/servidor 4898
Servidor escuchando en el puerto 4898. Ctrl-C para terminar.
```

El servidor se levanta en el puerto 4898 y queda en espera.

```
ricardo@servidor: ~/Proyecto-Arquitectura-Cliente-S...
~/Proyecto-Arquitectura-Cliente-Servidor
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ ./bin/cliente 192.168
.100.102 4898
Conectado al servidor 192.168.100.102:4898
Escribe un comando SQL por linea. SALIR para terminar.
SQL> S
```

El cliente se conecta desde la otra máquina a la IP del servidor.

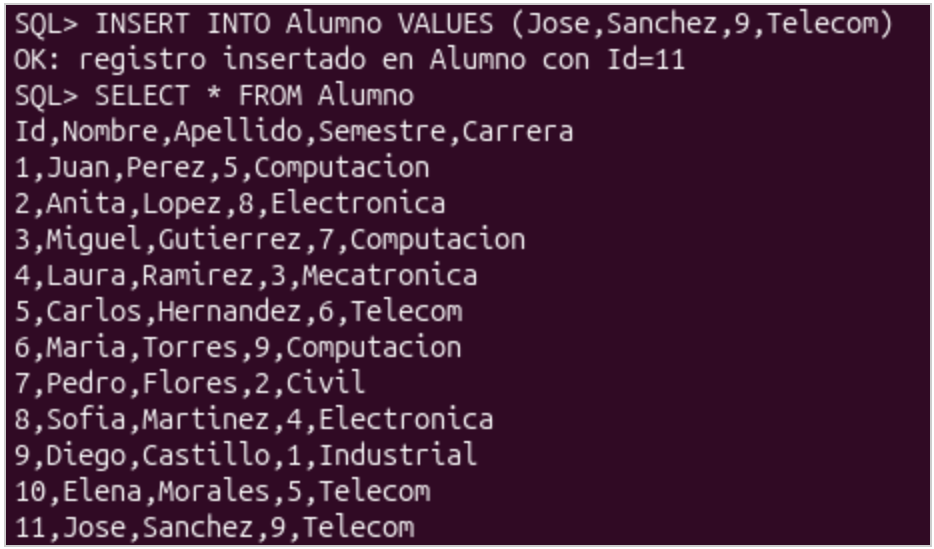
```
ricardo@servidor: ~/Proyecto-Arquitectura-Cliente-S...
~/Proyecto-Arquitectura-Cliente-Servidor
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$ ./bin/servidor 4898
Servidor escuchando en el puerto 4898. Ctrl-C para terminar.
02/11/2026 01:35:52 - Cliente 1 conectado desde 192.168.100.100:48370
```

El servidor registra la fecha, la hora y la IP del cliente conectado.

Ejecución de los comandos



SELECT * FROM Alumno muestra la cabecera y los diez registros de la tabla.



INSERT agrega el registro con Id 11 y el SELECT lo muestra al final.

```
SQL> UPDATE Alumno SET Apellido=Gomez WHERE Id=2
OK: 1 registro(s) actualizado(s) en Alumno
SQL> SELECT * FROM Alumno
Id,Nombre,Apellido,Semestre,Carrera
1,Juan,Perez,5,Computacion
2,Anita,Gomez,8,Electronica
3,Miguel,Gutierrez,7,Computacion
4,Laura,Ramirez,3,Mecatronica
5,Carlos,Hernandez,6,Telecom
6,Maria,Torres,9,Computacion
7,Pedro,Flores,2,Civil
8,Sofia,Martinez,4,Electronica
9,Diego,Castillo,1,Industrial
10,Elena,Morales,5,Telecom
11,Jose,Sanchez,9,Telecom
```

UPDATE cambia el apellido del registro con Id 2 y el SELECT lo confirma.

```
SQL> UPDATE Alumno SET Id=50 WHERE Id=2
ERROR: la columna Id es la llave primaria y no puede actualizarse
```

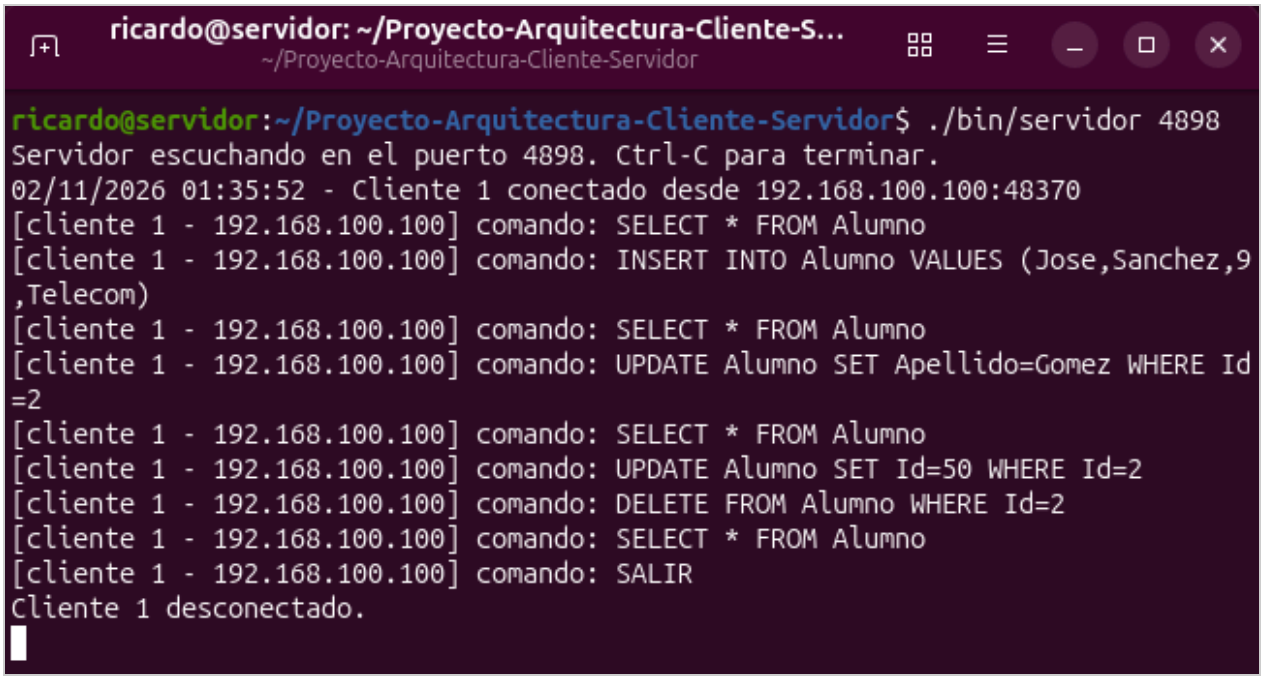
El intento de modificar la columna Id se rechaza por ser la llave primaria.

```
SQL> DELETE FROM Alumno WHERE Id=2
OK: 1 registro(s) eliminado(s) de Alumno
SQL> SELECT * FROM Alumno
Id,Nombre,Apellido,Semestre,Carrera
1,Juan,Perez,5,Computacion
3,Miguel,Gutierrez,7,Computacion
4,Laura,Ramirez,3,Mecatronica
5,Carlos,Hernandez,6,Telecom
6,Maria,Torres,9,Computacion
7,Pedro,Flores,2,Civil
8,Sofia,Martinez,4,Electronica
9,Diego,Castillo,1,Industrial
10,Elena,Morales,5,Telecom
11,Jose,Sanchez,9,Telecom
```

DELETE elimina el registro con Id 2 y el SELECT ya no lo muestra.

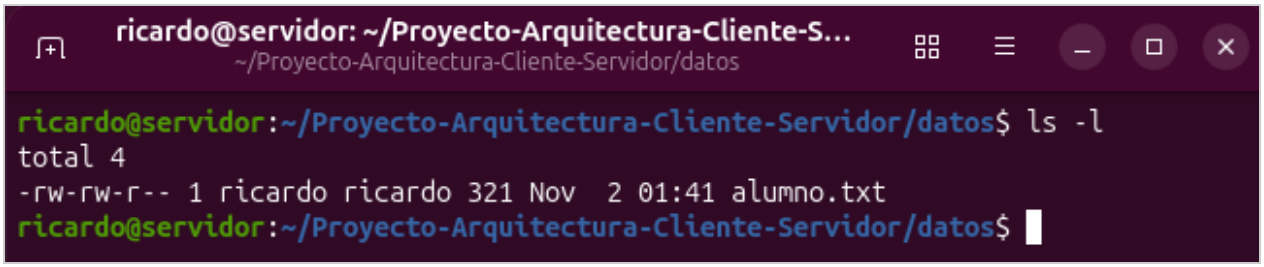
```
SQL> salir
Adios. Conexion cerrada por el servidor.
ricardo@servidor:~/Proyecto-Arquitectura-Cliente-Servidor$
```

SALIR recibe la despedida del servidor y el cliente termina.

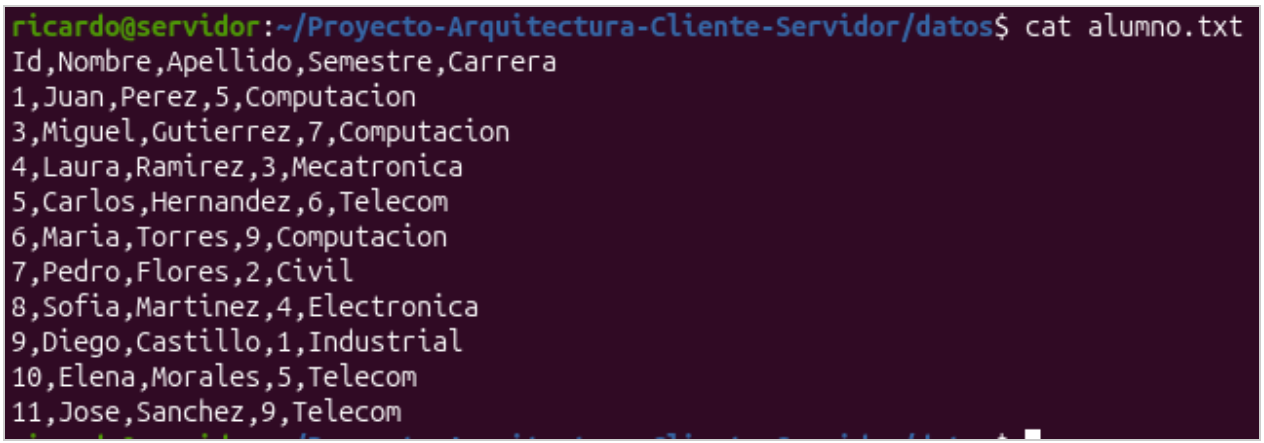


Bitácora del servidor con todos los comandos recibidos del cliente y su desconexión.

Estado final del archivo



El archivo alumno.txt fue modificado en la máquina servidor.



Contenido final del archivo: sin el registro con Id 2 y con el registro insertado con Id 11.

CONCLUSIONES

Arreola Santander Ricardo

Para resolver el proyecto se construyeron dos programas en C que se comunican por sockets TCP: un servidor que guarda la tabla en un archivo de texto y ejecuta los comandos, y un cliente que se conecta desde otra máquina para enviarlos. Al hacerlo se vio en la práctica la secuencia de llamadas de un servidor, que crea el socket, lo asocia a un puerto, escucha y acepta conexiones, y la del cliente, que resuelve la dirección y se conecta.

Un punto que resultó importante fue que TCP entrega un flujo de bytes y no mensajes separados, por lo que fue necesario acordar un formato: un comando por línea del lado del cliente y un carácter de fin de respuesta del lado del servidor. También se comprobó la utilidad de crear un proceso por cada cliente con fork, lo que permite atender varias conexiones a la vez, y la necesidad de bloquear el archivo para que dos procesos no lo escriban al mismo tiempo.

Con las pruebas entre dos máquinas virtuales se verificó que los cuatro comandos SQL funcionan sobre el archivo, que la columna Id se asigna de forma automática y no puede modificarse, y que el comando SALIR cierra la conexión de forma ordenada. El proyecto permitió unir en una sola aplicación los temas de procesos, sockets y manejo de archivos en Linux.

BIBLIOGRAFÍA Y FUENTES

• Román Zamitiz, C. A. *Arquitectura Cliente-Servidor. Tema 3: Sockets Internet en lenguaje C*. Facultad de Ingeniería, UNAM. Accedido el 1 de noviembre de 2026 de http://profesores.fi-b.unam.mx/carlos/acs/Tema-03-Sockets-INTERNET-C/index.html

• The Open Group. *The Open Group Base Specifications Issue 6, IEEE Std 1003.1*. Accedido el 1 de noviembre de 2026 de https://pubs.opengroup.org/onlinepubs/009696799/idx/index.html

• Stevens, W. R., Rago, S. A. *Advanced Programming in the UNIX Environment*, 3a edición, editorial Addison-Wesley, 2013. Referencia para el manejo de procesos, archivos y sockets en sistemas UNIX.

• Tanenbaum, A. S., Bos, H. *Sistemas Operativos Modernos*, 4a edición, editorial Pearson, 2016. Referencia para los conceptos de procesos y comunicación entre ellos.

• Kernighan, B. W., Ritchie, D. M. *El lenguaje de programación C*, 2a edición, editorial Prentice Hall, 1991.